

Advanced topics on Algorithms

Luciano Gualà

www.mat.uniroma2.it/~guala/

Algorithms for Big Data

Episode III

references:

- G.S. Manku, R. Motwani:

[Approximate Frequency Counts over Data Streams](#). VLDB (2002)

<https://www.vldb.org/conf/2002/S10P03.pdf>

- C. Demestrescu, I. Finocchi

[Algorithms for Data Streams](#)

<http://www.dei.unipd.it/~geppo/PrAvAlg/DOCS/DFchapter08.pdf>

Finding frequent items in a stream

An application of Sampling

The problem:

given a stream of elements, find the elements whose frequency is above a given threshold

Application domains:

- Data Base world
- Data Mining
- Network Monitoring
- ...

Data Base: iceberg queries

Choosing a good city for a trip...

SELECT: *City*; COUNT(*)

FROM: *Irish_Pubs*

GROUP BY: *City* HAVING: COUNT(*) $\geq T$



Data Mining: discovering association rules

I: set of items (products)

D: set of transactions

- a transaction: $T \subseteq I$

an **association rule** is an implication of the form

$X \Rightarrow Y$ (if you buy X then you also buy Y)

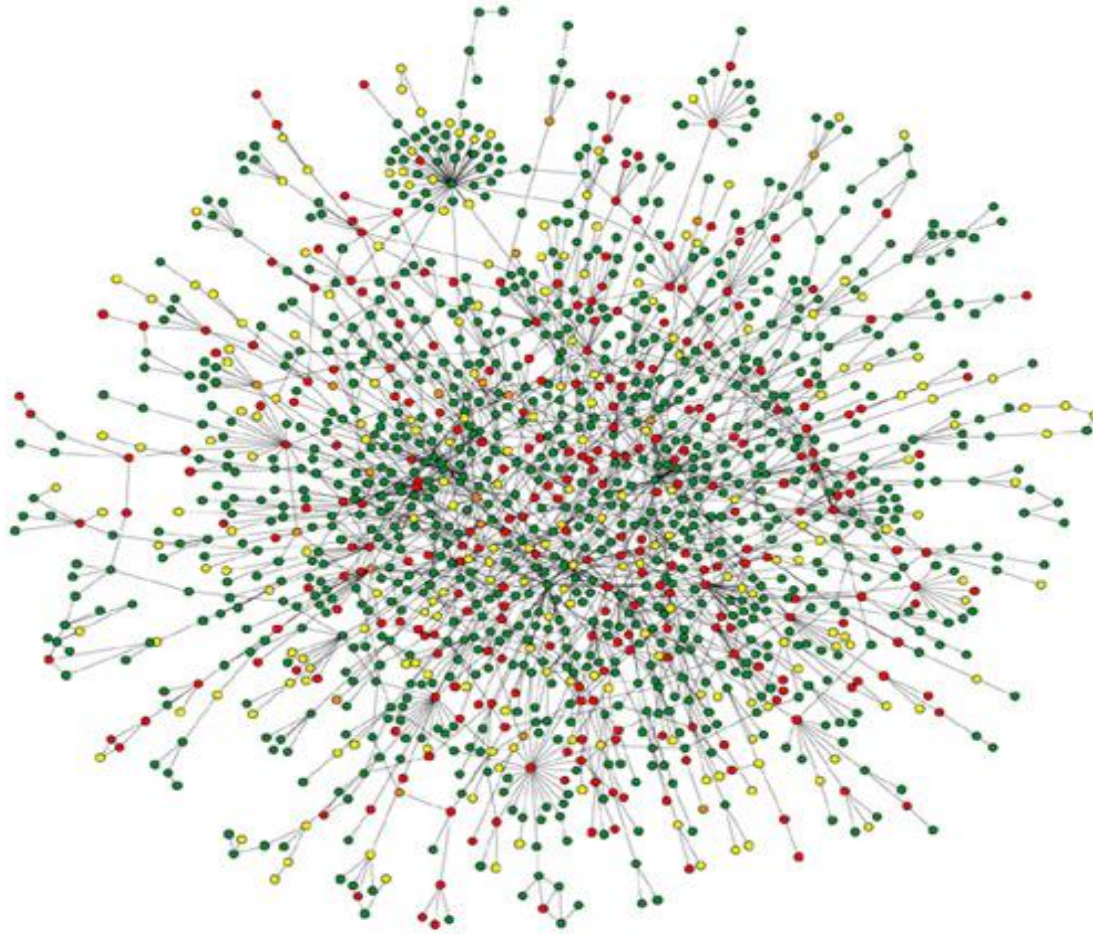
$X \Rightarrow Y$ holds in D with **confidence** c if $c\%$ of the transactions of D that contain X also contain Y

$X \Rightarrow Y$ has **support** s in D if $s\%$ of the transactions of D contain $X \cup Y$

goal: find association rules for D whose confidence and support are above certain thresholds

it reduces to the problem of finding frequent itemsets

Network Monitoring: Measurement and monitoring of network traffic



a **flow**: sequence of packets with the same source+destination addresses

goal: identifying **large flows**, i.e. flows sending more than a given threshold ($> s\%$ of the link capacity)

The problem

Given two parameters $0 < \epsilon < \phi < 1$, and a stream of n elements $x_1, x_2, \dots, x_n$, find:

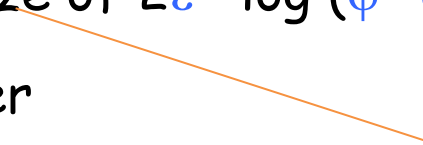
- all items whose frequency is at least ϕn (no false negative).
- no item with frequency smaller than $(\phi - \epsilon)n$.

Sticky sampling algorithm (Manku & Motwani, 2002)

- randomized
- meet the two goals with probability $1 - \delta$
- maintain a sample of expected size of $2\epsilon^{-1} \log(\phi^{-1} \delta^{-1})$

$0 < \delta < 1$: user-defined error parameter

of elements
in sample



notice: space is independent of the stream length n

The algorithm

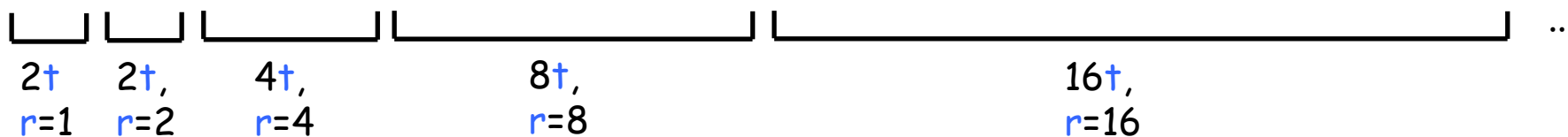
$$t = \varepsilon^{-1} \log(\varphi^{-1} \delta^{-1})$$

maintain a sample S : set of pairs $(x, f_e(x))$

- $f_e(x)$: estimation of the real frequency $f(x)$ of the element x

to handle potentially unbounded stream, computation proceeds in **windows**:

- each window has a **size** and a sampling **rate** r ;
- next window is double the size and the rate of the previous one;
- at the beginning S is empty and $r=1$.



in a give window, when the next stream element x comes:

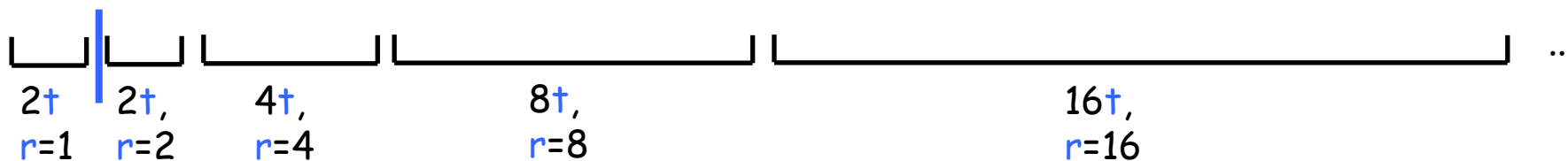
- if $x \in S$ then increase $f_e(x)$ by 1;
- otherwise, insert $(x, 1)$ in S with probability $1/r$.

The algorithm

$$t = \varepsilon^{-1} \log(\varphi^{-1} \delta^{-1})$$

whenever the sampling rate changes from r to $2r$ an adjusting step is performed:

goal: transform the state of S to the one it would have been in if the new rate $2r$ had been used from the beginning

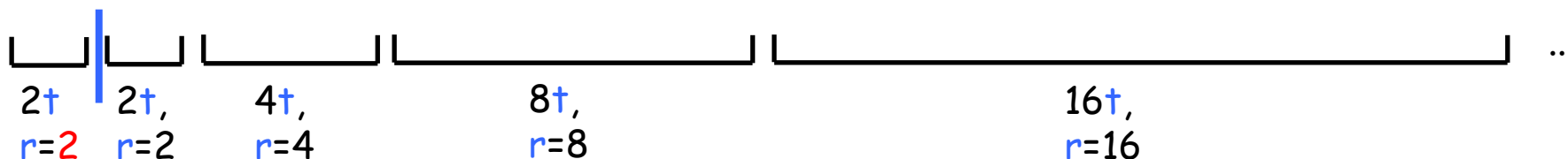


The algorithm

$$t = \varepsilon^{-1} \log(\varphi^{-1} \delta^{-1})$$

whenever the sampling rate changes from r to $2r$ an adjusting step is performed:

goal: transform the state of S to the one it would have been in if the new rate $2r$ had been used from the beginning

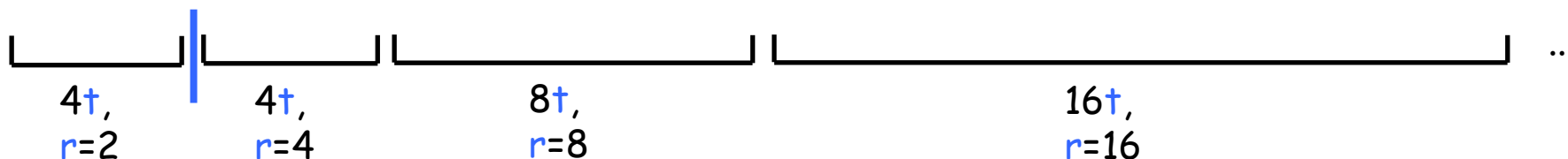


The algorithm

$$t = \varepsilon^{-1} \log(\varphi^{-1} \delta^{-1})$$

whenever the sampling rate changes from r to $2r$ an adjusting step is performed:

goal: transform the state of S to the one it would have been in if the new rate $2r$ had been used from the beginning

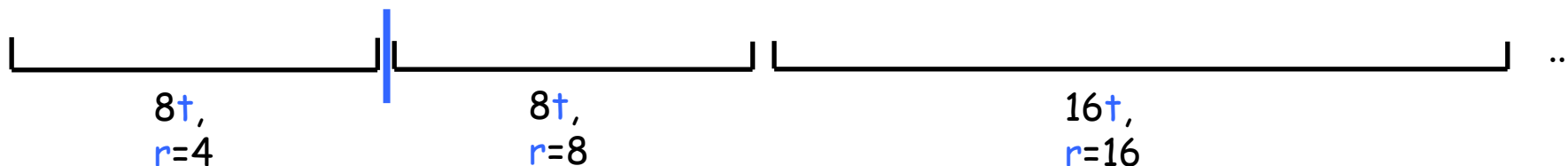


The algorithm

$$t = \varepsilon^{-1} \log(\varphi^{-1} \delta^{-1})$$

whenever the sampling rate changes from r to $2r$ an adjusting step is performed:

goal: transform the state of S to the one it would have been in if the new rate $2r$ had been used from the beginning

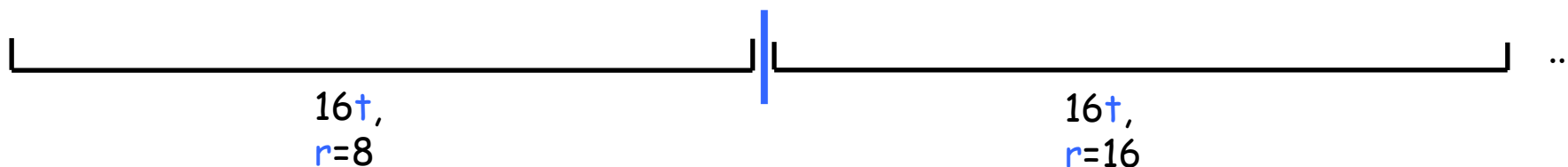


The algorithm

$$t = \varepsilon^{-1} \log(\varphi^{-1} \delta^{-1})$$

whenever the sampling rate changes from r to $2r$ an adjusting step is performed:

goal: transform the state of S to the one it would have been in if the new rate $2r$ had been used from the beginning

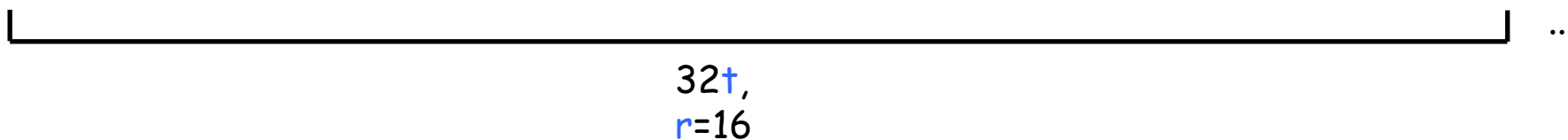


The algorithm

$$t = \varepsilon^{-1} \log(\varphi^{-1} \delta^{-1})$$

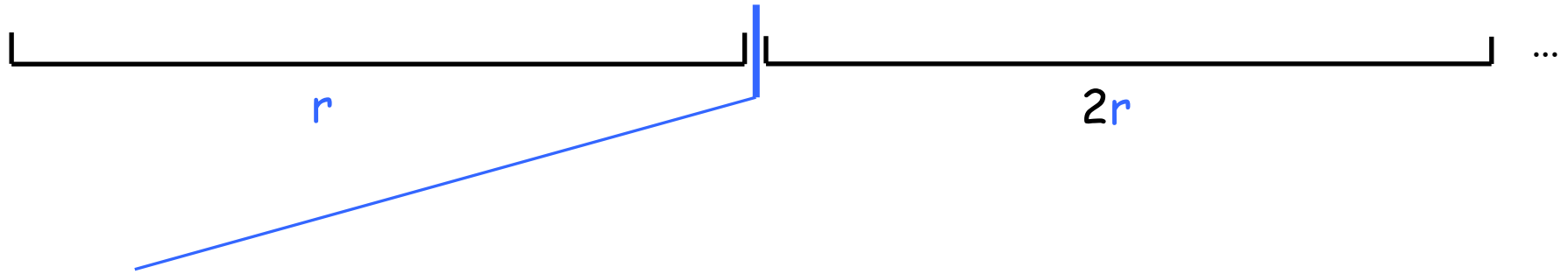
whenever the sampling rate changes from r to $2r$ an adjusting step is performed:

goal: transform the state of S to the one it would have been in if the new rate $2r$ had been used from the beginning



query(n): return all items in S with estimated frequency at least $(\varphi - \varepsilon)n$.

The algorithm: the adjusting step



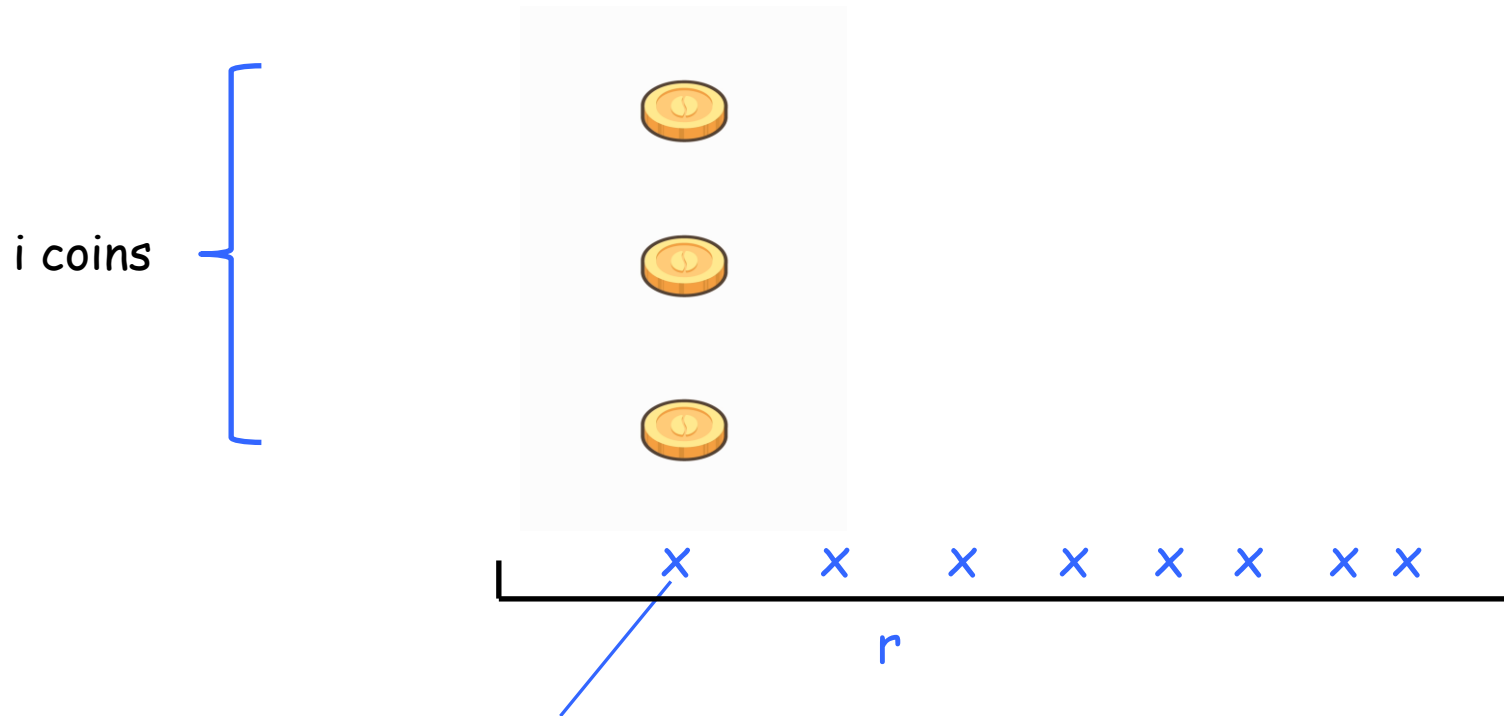
for each element $x \in S$:

- flip a fair coin
- if (Tail) then
 - repeatedly flip a coin with success probability of $1/(2r)$ until you get a success;
 - let k be the number of coin flips performed;
 - decrease $f_e(x)$ by k
 - if $f_e(x) \leq 0$ then remove x from S .

The adjusting step: the analysis

obs: r is always a power of 2 $\Rightarrow$ assume $r=2^i$

focus on a given element x and consider all its occurrences in the window

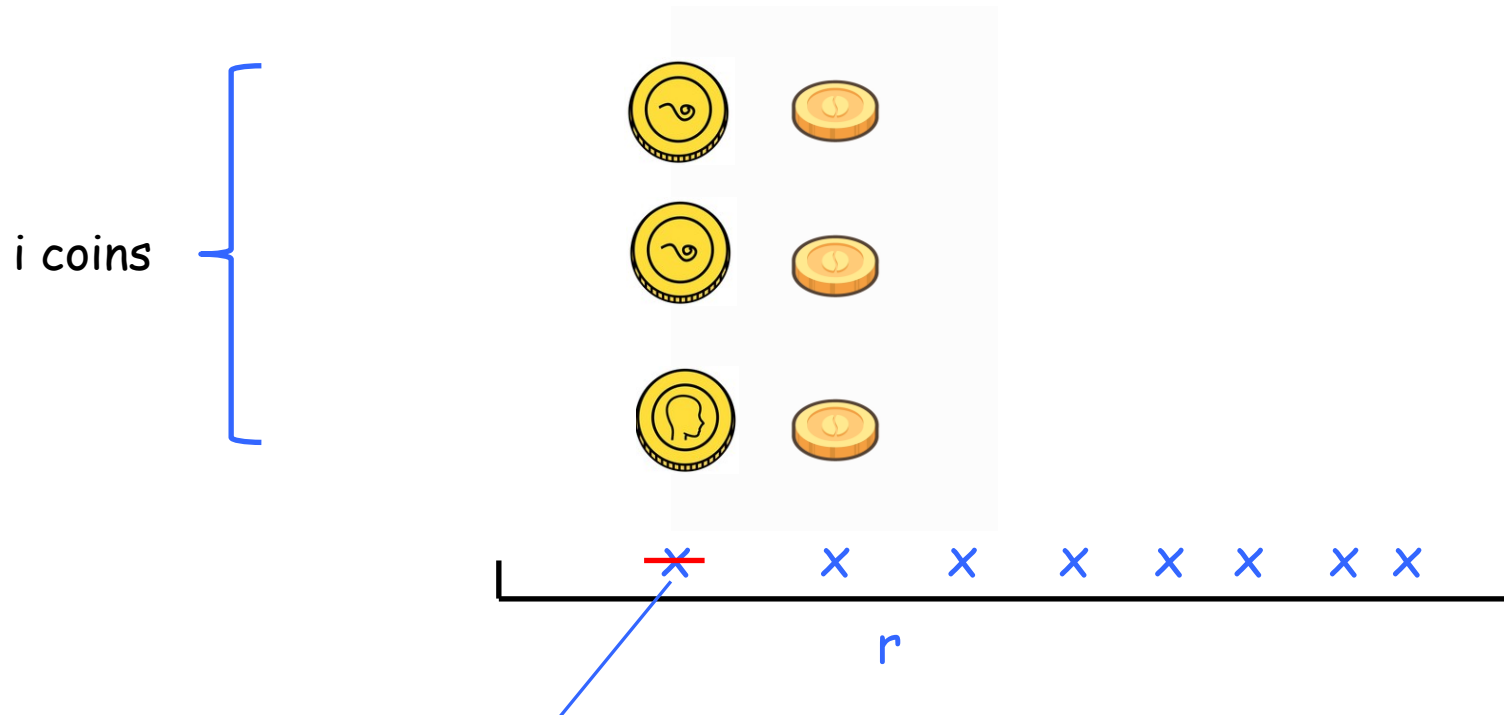


- first occurrence
- put in S with probability $1/r$

The adjusting step: the analysis

obs: r is always a power of 2 $\Rightarrow$ assume $r=2^i$

focus on a given element x and consider all its occurrences in the window

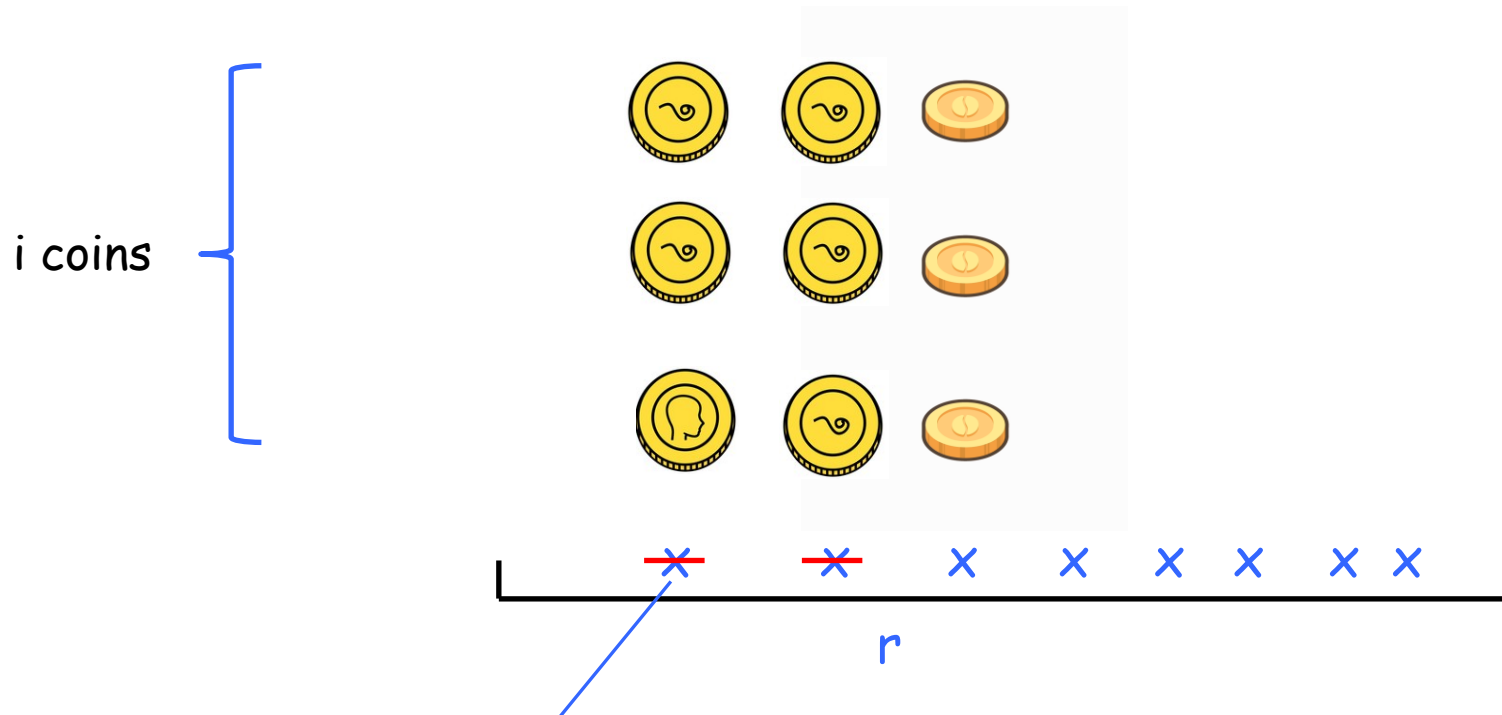


- first occurrence
- put in S with probability $1/r$

The adjusting step: the analysis

obs: r is always a power of 2 $\Rightarrow$ assume $r=2^i$

focus on a given element x and consider all its occurrences in the window

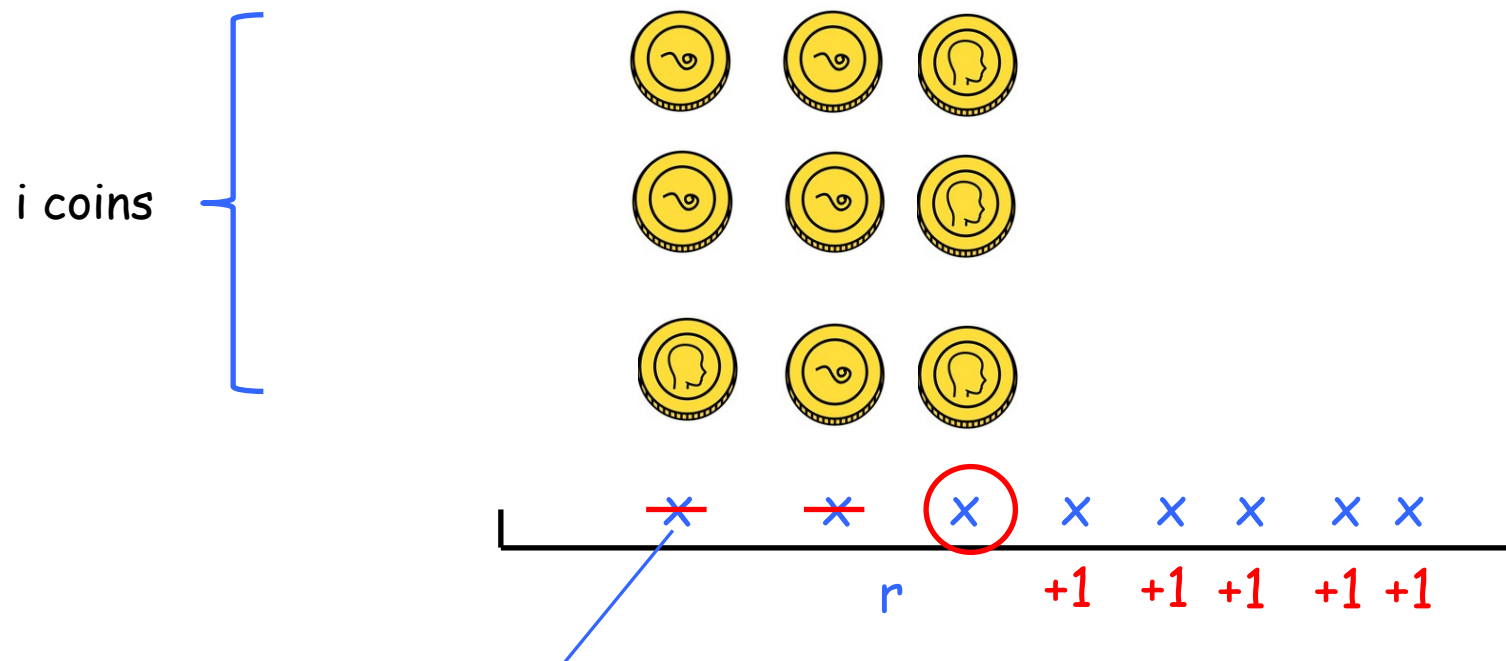


- first occurrence
- put in S with probability $1/r$

The adjusting step: the analysis

obs: r is always a power of 2 $\Rightarrow$ assume $r=2^i$

focus on a given element x and consider all its occurrences in the window



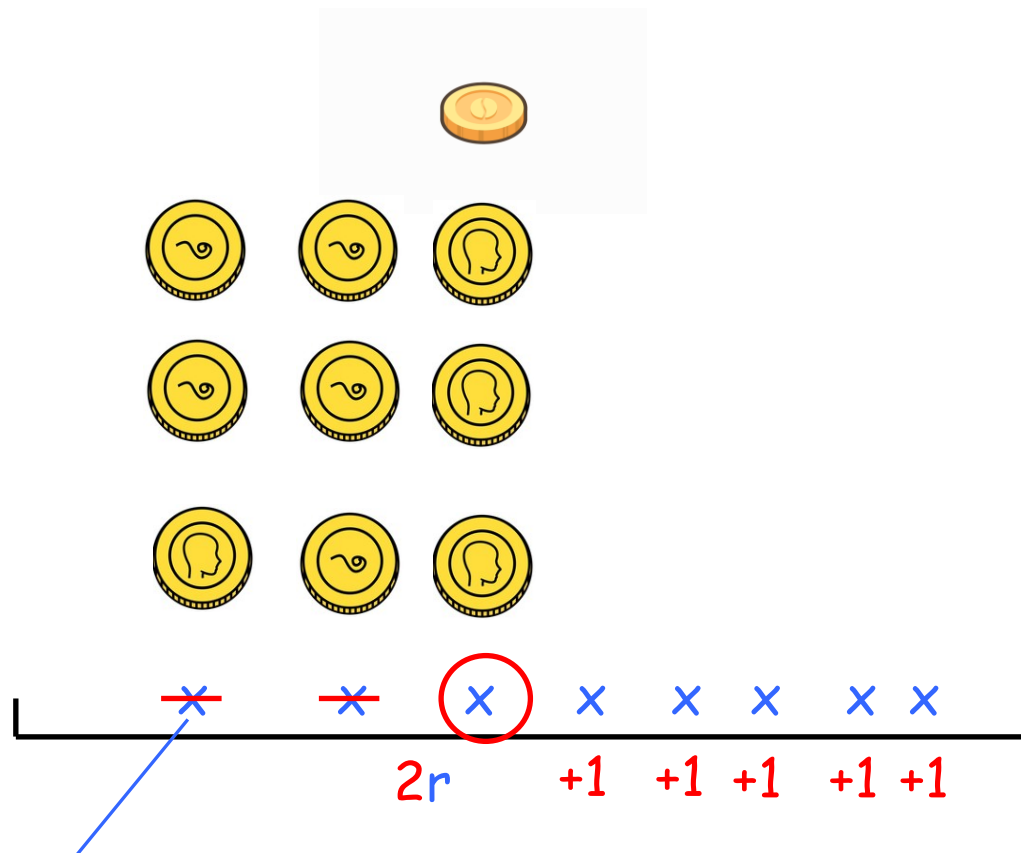
- first occurrence
- put in S with probability $1/r$

The adjusting step: the analysis

obs: r is always a power of 2 $\Rightarrow$ assume $r=2^i$

focus on a given element x and consider all its occurrences in the window

$i+1$ coins

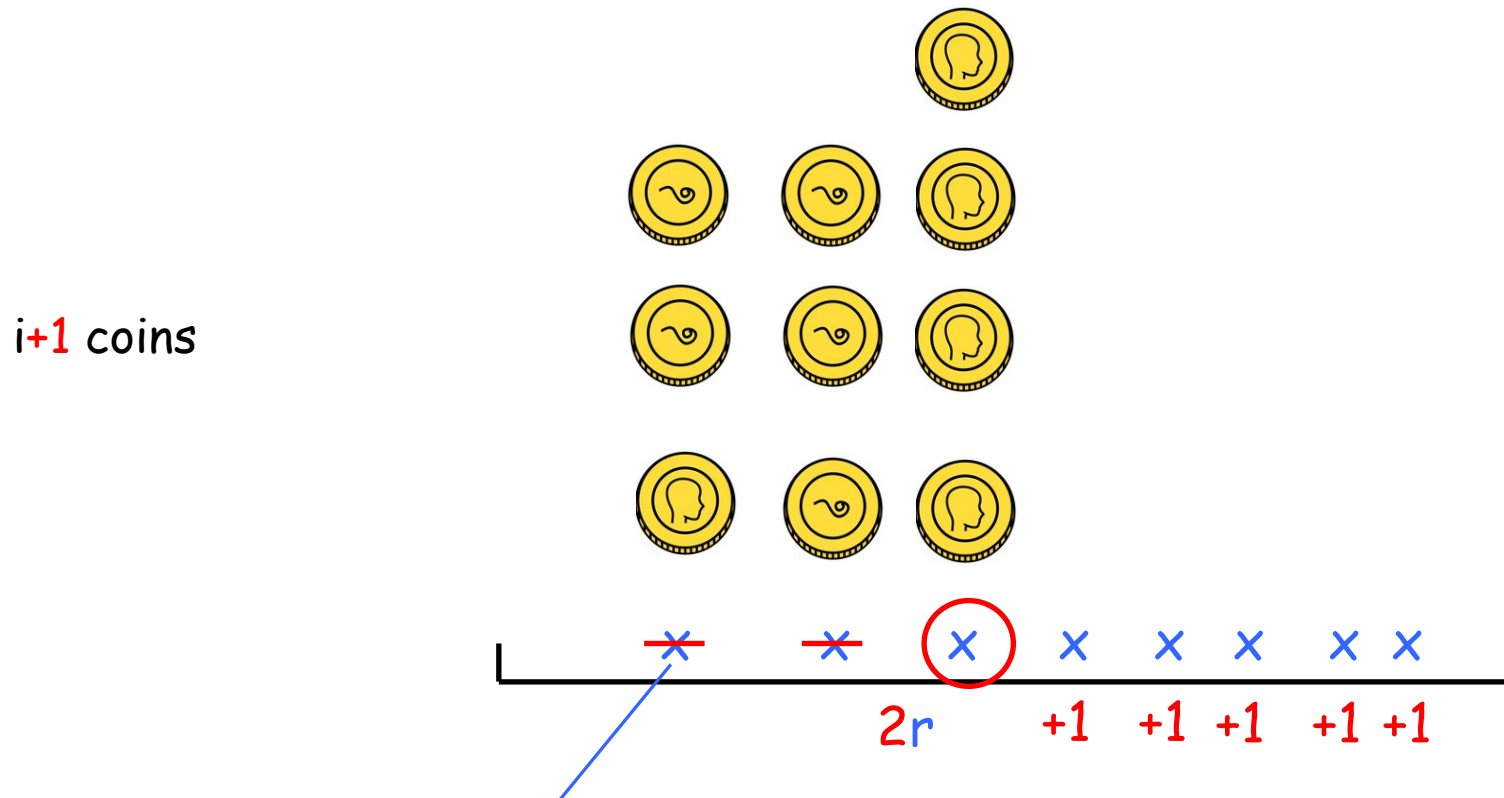


- first occurrence
- put in S with probability $1/r$

The adjusting step: the analysis

obs: r is always a power of 2 $\Rightarrow$ assume $r=2^i$

focus on a given element x and consider all its occurrences in the window



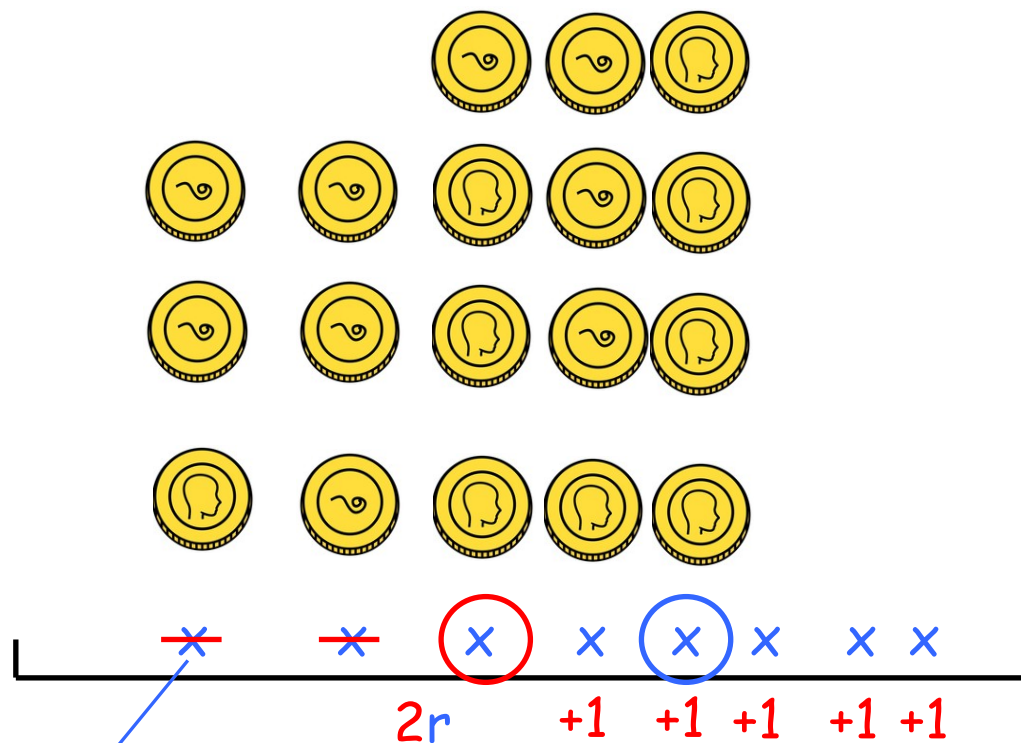
- first occurrence
- put in S with probability $1/r$

The adjusting step: the analysis

obs: r is always a power of 2 $\Rightarrow$ assume $r=2^i$

focus on a given element x and consider all its occurrences in the window

$i+1$ coins



- first occurrence
- put in S with probability $1/r$

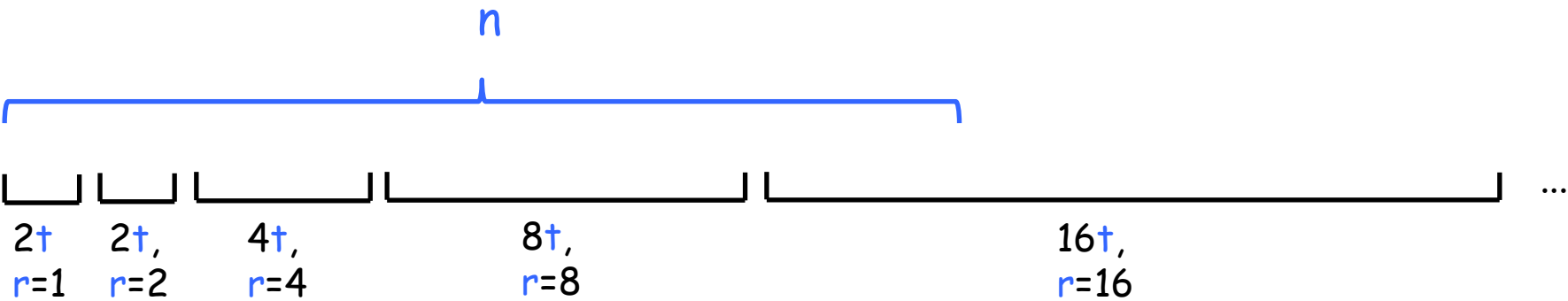
k (correction)

Lemma

Let n be the number of stream elements seen so far and assume that the current sample rate is r . Then $1/r \geq t/n$.

proof

$$n \geq t r$$



Theorem

For any $\varepsilon, \varphi, \delta \in (0,1)$, with $\varepsilon < \varphi$, sticky sampling solves the frequent items problem with probability at least $1 - \delta$ using a sample of expected size $2\varepsilon^{-1} \log(\varphi^{-1} \delta^{-1})$.

notice: $f_e(x) \leq f(x)$  algorithm never return an item with $f(x) < (\varphi - \varepsilon)n$.

let $y_1, \dots, y_k$, be the elements whose frequency is at least φn .

Clearly: $k \leq 1/\varphi$.

from previous
Lemma

$$\Pr[f_e(y_i) < (\varphi - \varepsilon)n] \leq (1 - 1/r)^{\varepsilon n} \leq (1 - \dagger/n)^{\varepsilon n} \leq e^{-\dagger \varepsilon}$$

$$\Pr[\exists \text{ false negative}] \leq \sum_{i=1}^k \Pr[y_i \text{ is not returned}] \leq \sum_{i=1}^k \Pr[f_e(y_i) < (\varphi - \varepsilon)n]$$

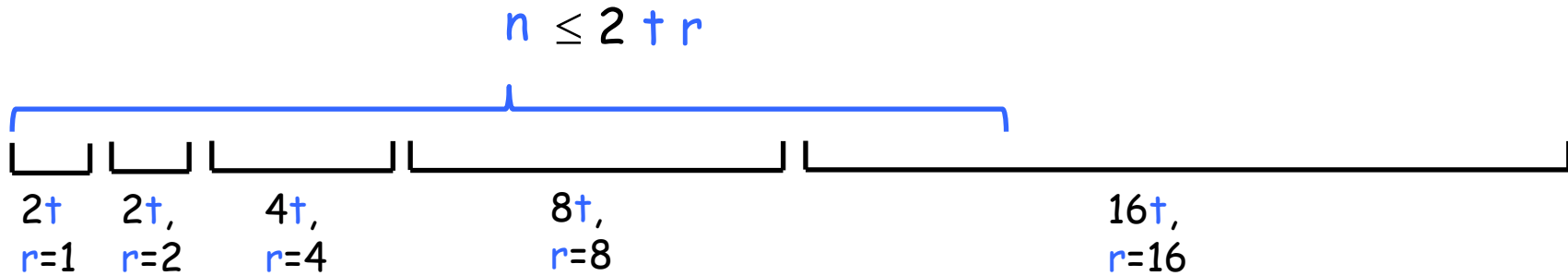
union
bound

$$\dagger = \varepsilon^{-1} \log(\varphi^{-1} \delta^{-1})$$

$$\leq k e^{-\dagger \varepsilon} \leq \frac{e^{-\dagger \varepsilon}}{\varphi} = \delta$$

what about the size of S ?

worst case: all stream elements are distinct



$$X_i \text{ r. v.} = \begin{cases} 1 & \text{if element } i \text{ is inserted in } S \\ 0 & \text{otherwise} \end{cases}$$

$$|S| = X = \sum_i X_i$$

$$E[X] = E\left[\sum_i X_i\right] = \sum_i \underbrace{E[X_i]}_{1/r} = n \cdot 1/r \leq 2t + r$$



Another result for the problem

In the paper

G.S. Manku, R. Motwani:

Approximate Frequency Counts over Data Streams. VLDB (2002)

Sticky sampling algorithm

- randomized
- meet the two goals with probability $1-\delta$
- maintain a sample of expected size of $2\epsilon^{-1} \log(\phi^{-1} \delta^{-1})$

$0 < \delta < 1$: user-defined error parameter

Lossy counting algorithm

- deterministic (meet the two goals with probability 1)
- maintain a sample of size of $O(\epsilon^{-1} \log(\epsilon n))$